Vélo'v : quel fichier pour quoi

Carte du projet, du ramassage des données à la prédiction. Mise à jour : 29 octobre 2026, fin du TP4 (Must 1 à 4).

Le parcours, étape par étape

1. Collecter — vraies données Vélo'v + météo
Python · DuckDB
`src/velov/ingest.py`
`data/velov.duckdb` (tables bronze_...)

▼

2. Nettoyer — bronze, silver, gold
dbt
`dbt/models/silver/*.sql` (nettoyage)
`dbt/models/gold/station_hourly.sql` (table d'entraînement)
`dbt/models/**/*.yaml` (tests, contrat) · `dbt/tests/*.sql` (tests sur mesure)
`dbt/profiles.yml` (où est la base)

▼

3. Contrôler — avant d'apprendre
Pandera
`src/velov/contracts.py`

▼

4. Apprendre — entraîner le modèle
scikit-learn
`src/velov/data.py` (données simulées, en attendant l'historique réel)
`src/velov/features.py` (heure, jour, météo)
`src/velov/train.py` (entraînement, seed, MAE, heures de pointe)
`models/model.joblib` + `models/metadata.json` (le paquet produit)

▼

5. Noter — le carnet de bord, sur la VM
MLflow
`src/velov/tracking.py` (un run par entraînement)
`compose.mlflow.yaml` · `docker/mlflow.Dockerfile` (le serveur)

▼

6. Choisir — quel modèle sert
MLflow · registre
`src/velov/promote.py` Must 3 (règle : même fenêtre, bat la persistance, bat le champion de 2 %)

▼

7. Répondre — aux questions des usagers
FastAPI · Pydantic · PostgreSQL
`src/velov/api/main.py` (les portes /health, /ready, /v1/predict)
`src/velov/api/schemas.py` (le formulaire et ses règles)
`src/velov/api/database.py` (garde chaque prédiction)

▼

8. Mettre en boîte — tourne pareil partout
Docker
`Dockerfile` (la boîte de l'API) · `.dockerignore`
`compose.yaml` (API + base + pgAdmin) · `scripts/build.sh` (tag version + commit)
`secrets/` (mots de passe, hors git) · `.env` (MLFLOW_DB_PASSWORD, hors git)

En continu — vérifier que rien ne casse
pytest · ruff · GitHub Actions · gitleaks
`tests/test_api.py` · `test_features.py` · `test_contract.py` · `conftest.py` (outils partagés)
`tests/test_model_quality.py` Must 4 (bat la persistance, même seed = même modèle)
`tests/test_promote.py` Must 3 (la règle, et le scénario sur un vrai registre)
`.github/workflows/ci.yml` Must 4 (le robot : tests, secrets, image)
`.gitleaksignore` (exceptions justifiées) · `pyproject.toml` (règles de ruff et pytest)

Les données (TP3) · Le modèle (TP1, TP4) · Le service (TP1, TP2) · La qualité (tout le long)

Où voir les résultats

Je veux voir...	Où	Comment
Poser une question à l'API	`http://localhost:8000/docs`	`docker compose up -d --build`, puis `POST /v1/predict`, Try it out, Execute
Les prédictions enregistrées	pgAdmin, `http://localhost:5050`	table `predictions` (README_EXECUTION, section 6)
Les runs d'entraînement	MLflow, `http://192.168.1.232:5000`	mode **Model training**, Experiments, `velov-availability` ; cocher des runs, puis Compare
Les versions et le champion	MLflow, onglet **Model registry**	`velov-availability` : alias @champion, @challenger, @previous-champion ; tag `promotion`
Le résultat de la CI	GitHub, onglet **Actions** du fork	une ligne par push, verte ou rouge ; cliquer pour le détail des 3 jobs
Les métriques du dernier modèle	`models/metadata.json`	`params`, `metrics`, `data_period`, `runtime`
Les données collectées	`data/velov.duckdb`	`duckdb data/velov.duckdb`, puis `SHOW TABLES;`
Les journaux du serveur MLflow	la VM	`ssh 192.168.1.232 "cd ~/velov-mlops && docker compose -f compose.mlflow.yaml logs --tail 60 mlflow"`

Les commandes du TP4

Commande	Ce qu'elle fait
`export MLFLOW_TRACKING_URI=http://192.168.1.232:5000`	Dit aux scripts où est le carnet MLflow (à refaire dans chaque nouveau terminal)
`python -m velov.data`	Crée 90 jours de données simulées (20 stations)
`python -m velov.train --seed 42 --mlflow`	Entraîne et note le run dans MLflow
`python -m velov.train --seed 42 --mlflow --register`	Pareil, et inscrit une nouvelle version au registre, avec l'alias @challenger
`python -m velov.promote`	Compare @challenger et @champion sur la même fenêtre, affiche la décision, ne change rien
`python -m velov.promote --apply`	Applique la décision (alias et tag `promotion`)
`ruff check . && ruff format --check . && pytest`	Les mêmes vérifications que la CI, en local
`MLFLOW_PORT=5000 docker --context vm compose -f compose.mlflow.yaml up -d`	(Re)déploie le serveur MLflow sur la VM depuis le Mac

La documentation

Fichier	Contenu
`README_EXECUTION.md`	Toutes les commandes, section par section : 1 à 7 lancer le projet ; 8 à 10 les données (TP3) ; 11 MLflow ; 12 la CI ; 13 la promotion
`docs/tp4_mlflow.md`	La stack MLflow : services, lancement, choix techniques (psycopg 3, allowed-hosts, CORS)
`docs/aide_memoire_commandes.pdf`	L'aide-mémoire des commandes (git, Docker, SSH, VM)
`docs/carte_du_projet.pdf`	Ce document
`docs/exigences.md`	Les exigences du projet (EX-01, EX-02…) et comment on les vérifie
`docs/adr/`	Les décisions d'architecture (ADR), avec leur contexte et les options écartées
`docs/model_card.md` · `docs/runbook.md`	La fiche du modèle ; quoi faire en cas d'incident

Document rédigé avec l'aide d'une IA (Claude, Anthropic), à partir des fichiers du dépôt.